WEB DEVELOPMENT WITH AI TOOLS · TASK 2 OF 2

E-commerce Store with Admin and Payments

Build a complete online store: customers browse, pay and track an order, and the owner manages products and orders.

Duration	1 week (7 days)
Level	Junior web developer: React, basic backend and database
Stack	Next.js or React with Node.js, a free database, and a payment provider in test mode (for example Stripe)
Why this task	Stores with payment integrations and easy product management are a core service, and they test real business logic.

> **Payment rule:** use test mode and test cards only. Never take real money or store card numbers. Prices and totals must be worked out on the server, never trusted from the browser.

THE SCENARIO

A made-up small brand sells 20 or more products and wants to stop taking orders by hand. Use free photos that match each product.

MUST HAVE

- **Catalogue:** categories, search, filters, sorting, pagination and a product page with variants such as size or colour.
- **Cart:** add, remove and change quantity. It stays after a refresh and shows subtotal, delivery and total.
- **Checkout:** guest checkout with an address form, plus test-mode card payment and cash on delivery. A failed payment must not create a paid order.
- **Stock:** reduced when an order is placed. A sold-out item or variant can't be bought, even by two buyers at the same moment.
- **Order tracking:** a confirmation page and a tracking page using order number and phone or email.
- **Admin area with a login:** add and edit products and variants, upload images, manage stock, change order status, and see sales and top products.
- **Quality:** mobile first, accessible forms, compressed images, a good title and description per page, and Product structured data.

HOW TO USE AI TOOLS (REQUIRED)

Build this with AI coding tools, such as Claude Code, Cursor, GitHub Copilot or ChatGPT. We want to see you lead the tool, not just accept what it writes.

- Plan first: write the pages, data model and steps in **PLAN.md** before asking the tool for code.
- Work in small steps. Run and read each change before you commit it.
- Keep **AI-NOTES.md**: the tools you used, your 5 most useful prompts, and at least **3 mistakes the AI made** and how you found and fixed them.
- Test by hand on a phone-sized screen. "The AI said it works" is not a test.

BONUS (ONLY AFTER THE MUST-HAVES)

Discount codes, an order email to the buyer, or end-to-end Playwright tests for the checkout.

DAY-BY-DAY PLAN

Day	Goal	Done means
1	PLAN.md, database design, project and first deploy	Data model written; live link exists
2	Catalogue, search, filters and product page	20 products browse well on a phone
3	Cart and checkout form	A cash on delivery order is saved
4	Test-mode payment and order status	Successful and failed payments behave correctly
5	Stock rules and order tracking	Sold-out and tracking cases work
6	Admin: products, orders, sales	Owner can run the store from admin
7	Edge-case testing, README, AI-NOTES, demo	Everything below is ready to send

WHAT TO SEND

- Live link and public GitHub repository with a README and setup steps. Give test admin login details by private message, not in the repository.
- PLAN.md and AI-NOTES.md.
- A short test list: at least 10 cases you tried, with results.
- A 3–5 minute video: buy with a test card, a failed payment, a sold-out item, tracking, then admin updating the order.

HOW IT IS SCORED

Area	Marks
Cart, checkout, totals and payment logic	30
Stock and order handling, including edge cases	15
Admin area	15
Design, user experience and mobile	15
Code quality and security (you can explain it)	10
AI tool use: plan, AI-NOTES, mistakes caught	10
README, Git history and demo	5

RULES

- No paid themes or ready-made templates. AI tools generate the code with you, so the design must still look made for this project.
- You must be able to explain every part of your code and change it live in the review call.
- Never put an API key or password in the repository or the video; use a **.env** file and commit **.env.example**.
- Commit every day with clear messages. One large commit at the end loses marks.
- If something is unfinished at the deadline, send what works and say what is missing.